

Differential Drive Robot

Introduction

Goal: Realizzazione di [DIFFERENTIAL DRIVE ROBOT](#) limitatamente all'analisi dei requisiti

Requirements

Costruire un sistema software che comanda un Differential Drive Robot (DDR) in modo che, partendo dalla posizione iniziale HOME, il robot si sposti lungo il perimetro di una stanza rettangolare vuota. Il DDR si deve fermare quando tornato in HOME.

Il DDR è anche sensibile ai dati rilevati da un Radar. Quando il Radar rileva un 'intruso' a una distanza minore di una distanza prefissata D_{MIN} :

- se il DDR si sta muovendo, il DDR si ferma fino a che il Radar non rileva più un 'intruso' così vicino
- se il DDR è fermo in HOME, il DDR ruota su sè stesso fino a che un 'intruso' così vicino scompare

Requirement analysis

L'analisi dei requisiti mira a chiarire le astrazioni software necessarie e a definire l'alfabeto dei messaggi e l'architettura logica iniziale del sistema, eliminando ogni ambiguità interpretativa dal testo dei requisiti tramite modelli computazionali formali.

Il sistema software preso in esame è composto dalle seguenti entità fondamentali:

HOME e la Stanza

Nei requisiti si parla di HOME come posizione iniziale in cui il DDR si trova all'interno della stanza e posizione finale del percorso lungo il perimetro. Per poter modellare la HOME è necessario prima formalizzare la stanza. Possiamo modellare la stanza come un rettangolo in un piano euclideo con base B e altezza H , dove entrambe le misure sono espresse in metri. Tuttavia, poiché il robot non è un punto ma occuperà un'area fisica che assumiamo quadrata di lato R , risulta necessario adottare una discretizzazione dello spazio in unità robotiche (celle). La stanza viene così mappata su una griglia bidimensionale dove ogni cella rappresenta un'area di dimensione $R \times R$.

HOME viene quindi formalizzata come una cella specifica e predefinita all'interno della griglia (l'angolo in alto a sinistra), atta a contenere interamente il robot.

Possiamo formalizzare questa struttura spaziale con il seguente codice Kotlin:

```
/**
 * Rappresentazione formale della Stanza come griglia computazionale.
 * Sistema di riferimento: asse x verso destra, asse y verso il basso.
 * * @property B larghezza della stanza in unità robotiche (numero di celle su x)
 * * @property H altezza della stanza in unità robotiche (numero di celle su y)
 * * @property R lato dell'unità robotica in metri (coincide con il lato di ogni cella)
 */
data class Stanza(
    val B: Int,
    val H: Int,
    val R: Double
){
    /** Dimensioni fisiche della stanza in metri */
    val larghezzaMetri: Double get() = B * R
    val altezzaMetri: Double get() = H * R

    /** Posizione HOME: cella iniziale predefinita (0, 0) */
    val home: Posizione = Posizione(0, 0)
}

/**
 * Coordinate discrete di una cella nella griglia spaziale.
 */
data class Posizione(val x: Int, val y: Int)
```

Radar

Il Radar è un'entità proattiva deputata al monitoraggio dell'intorno del DDR per rilevare eventuali "intrusi" a una distanza inferiore a D_{MIN} . La sua proprietà fondamentale è la generatione del dato informativo D (distanza in metri). Nel metamodello QAK viene formalizzato come un **attore**.

Per definire come questo dato sia accessibile al sistema, si profilano due opzioni di formalizzazione per la comunicazione:

Opzione 1: Modello ad Eventi (Push Communication)

Il radar comunica in autonomia la distanza rilevata tramite un evento asincrono, poiché non conosce (e non deve conoscere) i destinatari dell'informazione.

```
System ddrsystem

// ----- Definizione dei Messaggi -----
Event radar : distance(D) // D: distanza rilevata in metri (Double)

// ----- Definizione del Contesto -----
Context ctxddrsys ip [ host="localhost" port=8010 ]

// ----- Definizione dell'Attore -----
QActor radar context ctxddrsys {
    State s0 initial {
        println("$name | starting")
    }
    Goto rileva_distanza

    State rileva_distanza {
        // Rilevamento formale del dato D
    }
    Transition t0
        whenTime 1000 -> rileva_distanza
}
```

Opzione 2: Modello Request-Response (Pull Communication)

Il radar comunica la distanza rilevata esclusivamente come risposta a una specifica richiesta esplicita inviata dal sistema di controllo.

```
System ddrsystem

// ----- Definizione dei Messaggi -----
Request getDistance : getDistance(X) // Richiesta del sistema
Reply radarReply : distance(D) for getDistance // Risposta del radar

// ----- Definizione del Contesto -----
Context ctxddrsys ip [ host="localhost" port=8010 ]

// ----- Definizione dell'Attore -----
QActor radar context ctxddrsys {
    State s0 initial {
        println("$name | starting")
    }
    Goto attesa_richieste

    State attesa_richieste {
        // Il radar rimane in attesa di sollecitazioni esterne
    }
    Transition t0
        whenRequest getDistance -> handleRequest

    State handleRequest {
        // Risposta formale al mittente con il dato D
    }
    Goto attesa_richieste
}
```

Scelta di Analisi: Tra i due modelli, si preferisce l'**Opzione 1 (Modello ad Eventi)**. Trattandosi di un requisito di sicurezza (arresto d'emergenza in presenza di un intruso), l'uso di un evento asincrono garantisce che la notifica del pericolo sia immediata (reattività ottimale), evitando il ritardo computazionale e il sovraccarico introdotto dal polling continuo richiesto dall'Opzione 2.

Differential Drive Robot (DDR)

Il DDR viene modellato come un'entità puramente reattiva, priva di logica decisionale autonoma. Esso funge da mero attuatore passivo di comandi di movimento elementari ricevuti dal supervisore. In conformità con le specifiche e il software fornito dal committente, le sua capacità operative sono limitate a un set ristretto di azioni atomiche: traslazione lineare in avanti (passo di una cella) e rotazione assiale sul proprio asse di 90°.

Viene formalizzato come un attore che espone un'interfaccia di comandi (dispatch) e restituisce l'esito dell'azione atomica:

```
// ----- Definizione dei Messaggi per il DDR -----
Dispatch ddrCmd : ddrCmd(MOVE) // MOVE in { step, turnLeft, turnRight, stop }
Dispatch stepDone : stepDone(X) // Feedback di movimento completato con successo
Dispatch stepFail : stepFail(X) // Feedback di movimento fallito (es. ostacolo)

QActor ddr context ctxddrsys {
    State s0 initial {
        println("$name | attuatore DDR pronto")
    }
    Transition t0
        whenMsg ddrCmd -> gestisciMovimento

    State gestisciMovimento {
        // Interfaccia di ricezione passiva del comando atomico
    }
    Goto s0
}
```

Mind

Il sistema si occuperà del controllo e del coordinamento del DDR in conformità alle specifiche di progetto (navigazione perimetrale e gestione degli allarmi). Riceve il dato della distanza D dal Radar e lo confronta con la soglia critica D_{MIN} .

In questa fase di analisi dei requisiti, la struttura formale dell'attore Mind (così da noi definito per riferirci al sistema) viene definita specificando la sua interfaccia di ricezione degli eventi del radar e lo scheletro degli stati macroscopici necessari a soddisfare le specifiche del committente:

```
QActor mind context ctxddrsys {
    State s0 initial {
        println("$name | starting e pianificazione missione")
    }
    Goto eseguiMissionePerimetro

    State eseguiMissionePerimetro {
        // Struttura macro per scorrere le celle del perimetro
    }
    Transition t0
        whenEvent radar -> valutaIncolumita

    State valutaIncolumita {
        // Controllo astratto se D < DMIN per smistamento negli stati di emergenza
    }
    Goto eseguiMissionePerimetro

    State statoEmergenzaHalt {
        // Gestione Requisito: "Se il DDR si sta muovendo, si ferma"
    }

    State statoRotazioneSicurezza {
        // Gestione Requisito: "Se il DDR è fermo in HOME, ruota su se stesso"
    }
}
```

Dall'analisi dello stato `statoRotazioneSicurezza` emerge una **deduzione formale fondamentale** per risolvere un'ambiguità del testo: affinché la rotazione del robot in HOME possa far "scompare" l'intruso (facendo aumentare la distanza $D > D_{MIN}$), il **Radar deve essere necessariamente montato a bordo del robot ed essere di tipo direzionale**. Se fosse un radar fisso nell'ambiente o omnidirezionale, la rotazione del robot su se stesso non varierebbe il flusso di dati del sensore, bloccando il sistema in un ciclo infinito.

Problem analysis

Test plans

Project

Testing

Deployment

Maintenance